



LINFO1104

Professeur Van Roy Peter

Rapport Projet Oz

Siméon Vanden Bulcke et Lucas Jacques

Mai 2025

Table des matières

1	Introduction	2
1.1	ExecuteBlockchain	2
1.2	AddBlockToBlockchain	2
2	Résultats Blockchain	3
2.1	Etat final	3
2.2	Schéma blockchain	3
2.3	Phrase secrète	3
3	Extensions	4
3.1	L'effort n'est pas gratuit	4
3.1.1	Implémentation	4
3.2	Denylist	5
3.2.1	Implémentation	5

1 Introduction

Dans le cadre du cours LEPL1101, il a été demandé d'implémenter un système de blockchain et de résoudre le mystérieux code de Sherlock Holmes.

1.1 ExecuteBlockchain

Pour `ExecuteBlockchain`, la procédure `ExecuteBlockchainHelper` a été créée sous forme de boucle récursive. Deux fonctions sont importantes : `AddTransactionToBlock` et `ValidateBlock`.

`AddTransactionToBlock` valide et ajoute les transactions au bloc courant. La validation est effectuée par `ValidateTransaction`, appelée à l'intérieur de cette même procédure, qui vérifie notamment que le hash est correct, que la transaction ne dépasse pas le `max_effort` et que le sender possède suffisamment d'argent sur son compte. Si la transaction est valide, `ApplyTransaction` est alors appelée pour effectuer les additions et soustractions sur les comptes des utilisateurs.

La procédure `ExecuteBlockchainHelper` examine ensuite si la transaction appartient au bloc courant grâce au `block_number` ou si elle doit déclencher la création d'un nouveau bloc. Dans ce second cas, le bloc en cours est finalisé via `FinalizeBlock`, qui inverse la liste des transactions et calcule son hash. Si le bloc est valide selon `ValidateBlock`, il est ajouté à la blockchain. Sinon, il est ignoré et l'exécution continue. Cette tolérance aux blocs invalides permet d'éviter qu'une erreur n'arrête complètement le système.

```
1 proc {ExecuteBlockchainHelper Transactions State CurrentBlock Blockchain FinalState FinalBlockchain}
2   case Transactions of nil then
3     FinalizedBlock = {FinalizeBlock CurrentBlock}
4   in
5     if {ValidateBlock FinalizedBlock {LastBlock Blockchain}} then
6       FinalBlockchain = {AddBlockToBlockchain FinalizedBlock Blockchain}
7     else
8       FinalBlockchain = Blockchain
9     end
10    FinalState = State
11  []Ti|Rest then
12    if Ti.block_number == CurrentBlock.number then
13      NewBlock NewState
14    in
15      {AddTransactionToBlock Ti CurrentBlock State NewBlock NewState}
16      {ExecuteBlockchainHelper Rest NewState NewBlock Blockchain FinalState FinalBlockchain}
17    else
18      FinalizedBlock NewBlockchain NewBlock
19    in
20      FinalizedBlock = {FinalizeBlock CurrentBlock}
21      if {ValidateBlock FinalizedBlock {LastBlock Blockchain}} then
22        NewBlockchain = {AddBlockToBlockchain FinalizedBlock Blockchain}
23      else
24        NewBlockchain = Blockchain
25      end
26      NewBlock = {BuildBlock {LastBlock NewBlockchain}}
27      {ExecuteBlockchainHelper Transactions State NewBlock NewBlockchain FinalState FinalBlockchain}
28    end
29  end
30 end
```

1.2 AddBlockToBlockchain

Cette fonction insère les blocs en queue de liste plutôt qu'en tête. De cette manière, l'ordre chronologique est respecté et la lecture de la blockchain est logique. Cet ordre est essentiel pour la cohérence de la chaîne, puisque chaque bloc référence le hash du bloc précédent via `previousHash`.

```
1 fun {AddBlockToBlockchain Block Blockchain}
2   case Blockchain of nil then
3     Block|nil
4   []B|nil then
5     B|Block|nil
6   []B|Rest then
7     B|{AddBlockToBlockchain Block Rest}
8   end
9 end
```

2 Résultats Blockchain

2.1 Etat final

Cette table présente l'état final des comptes après l'exécution de la blockchain, en indiquant pour chaque adresse son solde restant ainsi que son nonce (nombre de transactions effectuées en tant qu'expéditeur).

TABLE 1 – État final des comptes après exécution de la blockchain

Adresse	Solde	Nonce
11	97 415	2
14	70 294 080	8
15	94 650 918	6
32	5 141 804	7
37	1 239 534 151	0
49	463 119	1
60	36 160	3
61	221 043	4
66	480 590	2
73	58 497	4
86	918 173	2
87	210 757	1
100	579 432	3

2.2 Schéma blockchain

Deux fichiers ont été donnés : `genesis.txt` et `transactions.txt`. Le premier contient l'état initial des comptes avec quelques utilisateurs. Le deuxième contient les transactions effectuées. Dans ces transactions, on retrouve le `block_number`, le `nonce`, le `hash`, le `sender`, le `receiver`, la `value` et le `max_effort`. Notons que, même lorsque les transactions sont déjà affectées à un bloc, si toutes les conditions ne sont pas validées, la transaction est ignorée. Un nouveau bloc est créé lorsqu'une transaction arrive avec un `block_number` différent du bloc courant.

Par exemple, la transaction `block_number:5`, `sender:37`, `value:1239534151` est rejetée dès la vérification du hash dans `ValidateTransaction` : le hash attendu vaut

$$(1 + 37 + 14 + 1\,239\,534\,151) \bmod 10^6 = 534\,203$$

mais le champ `hash` de la transaction vaut `1 239 534 203` $\neq$ `534 203`. Même si ce contrôle avait passé, la transaction aurait été rejetée pour deux raisons supplémentaires : son effort calculé ($2^0 + 2^1 + \dots + 2^9 = 1023$) dépasse son `max_effort` de 1000, et la somme des efforts du bloc 5 aurait largement dépassé la limite de 300.

Ceci explique pourquoi certains blocs ne contiennent pas le même nombre de transactions que d'autres.

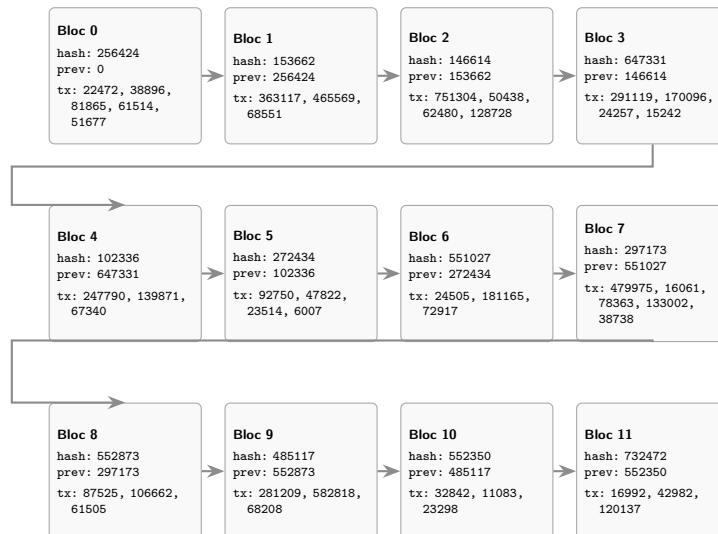


FIGURE 1 – Représentation de la blockchain finale (12 blocs). Chaque bloc affiche son numéro, son hash, le hash du bloc précédent (`prev`) et les hashes de ses transactions (`tx`).

2.3 Phrase secrète

La phrase secrète obtenue par décodage de la blockchain finale est : `prof peter van royiarty is behind oz`

3 Extensions

3.1 L'effort n'est pas gratuit

L'extension « L'effort n'est pas gratuit » est une extension permettant de calculer le coût qu'un utilisateur doit payer lorsqu'une transaction est exécutée. En effet, l'exécution d'une transaction nécessite un certain effort, auquel est associé un coût. Ce coût est fixé à une unité monétaire par unité d'effort. Dans le cas où l'utilisateur ne dispose pas d'un solde suffisant pour couvrir le coût de l'exécution, la transaction est annulée. Dans le cas contraire, la transaction est exécutée et le coût correspondant est déduit du compte de l'utilisateur.

3.1.1 Implémentation

Pour la création de cette extension, le fichier `BaseModule.oz` a été copié dans un nouveau fichier nommé `LEffortNEstPasGratuit.oz`, afin d'avoir une vue claire des endroits nécessitant des adaptations pour l'implémentation de cette extension. Les deux fichiers sont presque identiques, la principale différence réside dans la gestion de l'effort en tant que frais de transaction. Dans `BaseModule.oz`, l'effort n'a aucun impact sur les soldes des utilisateurs. Les deux implémentations ne diffèrent qu'en deux endroits :

TABLE 2 – Différences entre `BaseModule` et `LEffortNEstPasGratuit`

Fonction	BaseModule	LEffortNEstPasGratuit
<code>ValidateTransaction</code> (ligne 68)	<code>value > balance</code>	<code>(value + effort) > balance</code>
<code>ApplyTransaction</code> (ligne 102)	<code>balance - value</code>	<code>balance - (value + effort)</code>

L'effort est bien déduit des balances dans `LEffortNEstPasGratuit`, mais aucune transaction ne passe de valide à invalide à cause du coût supplémentaire de l'effort donc les deux versions restent les mêmes, la seule différence réside dans les balances totales des utilisateurs qui sont plus faibles que dans `BaseModule`.

TABLE 3 – Récapitulatif de la blockchain finale

Bloc	Hash	PreviousHash	Transactions (hash)
0	256424	0	22472, 38896, 81865, 61514, 51677
1	153662	256424	363117, 465569, 68551
2	146614	153662	751304, 50438, 62480, 128728
3	647331	146614	291119, 170096, 24257, 15242
4	102336	647331	247790, 139871, 67340
5	272434	102336	92750, 47822, 23514, 6007
6	551027	272434	24505, 181165, 72917
7	297173	551027	479975, 16061, 78363, 133002, 38738
8	552873	297173	87525, 106662, 61505
9	485117	552873	281209, 582818, 68208
10	552350	485117	32842, 11083, 23298
11	732472	552350	16992, 42982, 120137

TABLE 4 – Comparaison des soldes finaux entre `BaseModule` et `LEffortNEstPasGratuit`

Adresse	BaseModule	LEffortNEstPasGratuit	Différence
11	97 415	97 353	-62
14	70 294 080	70 293 640	-440
15	94 650 918	94 650 668	-250
32	5 141 804	5 141 523	-281
49	463 119	463 088	-31
60	36 160	36 035	-125
61	221 043	220 919	-124
66	480 590	480 496	-94
73	58 497	58 341	-156
86	918 173	918 127	-46
87	210 757	210 726	-31
100	579 432	579 275	-157

La phrase secrète obtenue par décodage de la blockchain finale est toujours : `prof peter van royiarty is behind oz`

3.2 Denylist

L'extension Denylist permet de créer une liste d'utilisateurs dont les transactions envoyées sont invalidées. La condition pour faire partie de cette liste d'avoir au moins 3 transactions traitées (tentatives incluses) au sein d'un même bloc. Néanmoins, les transactions déjà envoyées avant l'ajout à la liste sont tout de même prises en compte si elles ont été validées précédemment.

3.2.1 Implémentation

Pour créer cette extension, le fichier `BaseModule.oz` a été copié dans un nouveau fichier nommé `Denylist.oz`. Ce qui permettait d'identifier plus facilement les parties du code à modifier pour implémenter cette extension.

La structure des utilisateurs a été modifiée avec l'ajout de deux champs : `txCount`, qui correspond au nombre de transactions envoyées dans le bloc courant et `denied`, une variable booléenne indiquant si l'utilisateur est blacklisté ou non. Une fois ces champs ajoutés, une condition supplémentaire a été intégrée dans `ValidateTransaction` :

```
1 elseif State.(Transaction.sender).denied then false
```

Il a ensuite fallu compter le nombre de transactions par bloc. Deux lignes ont été ajoutées. La première incrémente le nombre de transactions envoyées et la seconde met le champ `denied` à `true` si un utilisateur envoie au moins trois transactions dans un même bloc. Ce qui permet donc de bloquer une éventuelle 4e transaction.

```
1 UpdatedSender1 = {AdjoinAt State.(NewTransaction.sender) txCount txCount + 1}
2 UpdatedSender2 = if UpdatedSender1.txCount >= 3 then {AdjoinAt UpdatedSender1 denied true}
3 else UpdatedSender1 end
```

Une nouvelle fonction, `ResetTxCounts`, remet les compteurs de transactions à zéro lorsqu'un nouveau bloc est créé. Le champ `denied` n'est cependant pas réinitialisé, puisque le bannissement est permanent. Enfin, `ResetTxCounts` est appelée lors du passage à un nouveau bloc dans `ExecuteBlockchainHelper`.

TABLE 5 – Récapitulatif de la blockchain finale

Bloc	Hash	PreviousHash	Transactions (hash)
0	256424	0	22472, 38896, 81865, 61514, 51677
1	153662	256424	363117, 465569, 68551
2	146614	153662	751304, 50438, 62480, 128728
3	356212	146614	170096, 24257, 15242
4	743877	356212	247790, 139871
5	913975	743877	92750, 47822, 23514, 6007
6	168063	913975	181165, 72917
7	914209	168063	479975, 16061, 78363, 133002, 38738
8	169909	914209	87525, 106662, 61505
9	519335	169909	281209, 68208
10	563270	519335	32842, 11083
11	743392	563270	16992, 42982, 120137

Les adresses 15, 32 et 73 sont bannies dans Denylist car elle faisaient au moins 3 tentatives de transactions au sein d'un même bloc.

TABLE 6 – Comparaison des soldes finaux entre `BaseModule` et `Denylist`

Adresse	BaseModule	Denylist	Différence
11	97 415	97 415	0
14	70 294 080	70 226 791	-67 289
15	94 650 918	94 043 702	-607 216
32	5 141 804	6 107 292	+965 488
37	1 239 534 151	1 239 534 151	0
49	463 119	463 119	0
60	36 160	36 160	0
61	221 043	221 043	0
66	480 590	457 435	-23 155
73	58 497	81 652	+23 155
86	918 173	918 173	0
87	210 757	210 757	0
100	579 432	288 449	-290 983

La phrase secrète obtenue par décodage de la blockchain finale est : `prof petezpc h g qh gp ejzjwx xi`